UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS Y COMPUTACIÓN
CARRERA DE INGENIERÍA EN SISTEMAS

SISTEMA PARA ADMINISTRACIÓN DEL ASILO DE ANCIANOS "CABEZA DE ALGODÓN"

FASE 2 - DISEÑO

Curso: Análisis de Sistemas ll

Catedrático: Ing. Ángel Atilio Maltez

Estudiante: Maria Yolanda Ajiataz Son

Carné: 3090-23-22955

Fecha: 27 septiembre de 2026

Introducción

Esta segunda fase toma como base el análisis realizado en la Fase 1 del Sistema Cabeza de Algodón. El sistema se plantea como una aplicación web modular respaldada por MySQL. En esta fase se define la arquitectura de software, los componentes principales, la separación de responsabilidades por capas, un microservicio sencillo de notificaciones y una propuesta de mejora de experiencia de usuario aplicada al módulo de registro de ancianos.

Objetivos de la fase de diseño

- Definir una arquitectura por capas que separe presentación, lógica de negocio, acceso a datos y entidades.
- Diseñar componentes y clases coherentes con los requerimientos definidos en la Fase 1.
- Seleccionar e integrar un microservicio sencillo y útil para el proyecto.
- Analizar la usabilidad del módulo de registro de ancianos y proponer mejoras de UX.
- Implementar al menos una mejora visible y documentar el antes y después.
- Establecer lineamientos de calidad que puedan ser revisados por SQA.

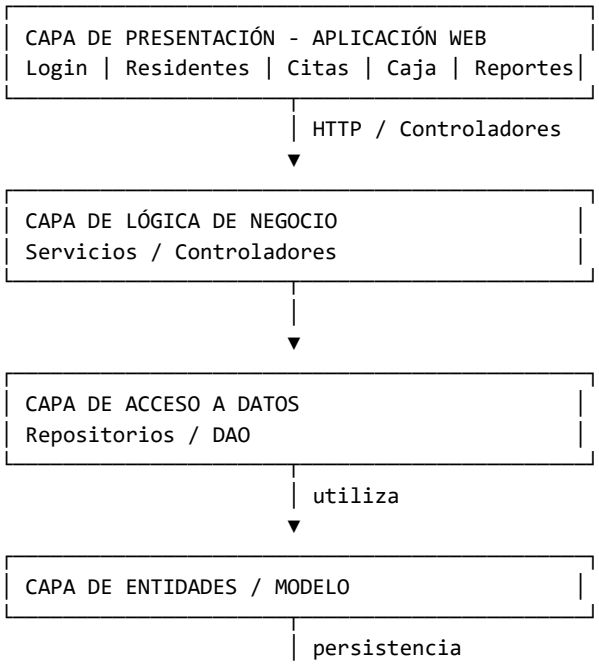
Arquitectura propuesta

Se utilizará una arquitectura por capas. La capa de presentación será web y no tendrá acceso directo a la base de datos. Toda operación iniciada desde la interfaz deberá pasar por la capa de lógica de negocio. Esta validará reglas y coordinará las operaciones con la capa de acceso a datos. Las entidades representan los objetos del dominio y son utilizadas por las capas internas.

Capas del sistema

Capa	Responsabilidad	Ejemplos
Presentación Web	Mostrar formularios, capturar datos, presentar mensajes y consumir únicamente servicios/controladores.	Login, Residentes, Citas, Caja, Reportes
Lógica de negocio	Aplicar validaciones y reglas del negocio; coordinar casos de uso.	PacienteService, FamiliarService, SolicitudService
Acceso a datos	Ejecutar operaciones CRUD y consultas contra MySQL.	PacienteRepository, FamiliarRepository
Entidades / Modelo	Representar los objetos principales del dominio.	Paciente, Familiar, FichaMedica, SolicitudCita
Base de datos	Persistir información con integridad referencial.	MySQL / AsiloDB

Diagrama lógico de arquitectura



▼
MySQL

Regla de comunicación

La presentación web se comunicará únicamente con la lógica de negocio. No se permitirá que una pantalla ejecute SQL ni utilice directamente repositorios. Esto reduce el acoplamiento y facilita mantenimiento, pruebas y cambios futuros.

Microservicio seleccionado: notificaciones por correo

Se propone un microservicio de notificaciones por correo. Su función es recibir una solicitud de envío desde la lógica de negocio y encargarse del correo dirigido al familiar. Es pequeño, independiente y útil porque el sistema debe informar al familiar cuando se crea una solicitud médica y cuando se asignan datos de atención.

Flujo propuesto:

- El médico general crea una solicitud médica desde la aplicación web.
- SolicitudService valida y registra la solicitud.
- La lógica de negocio envía al microservicio los datos mínimos de la notificación.
- El microservicio genera y envía el correo mediante SMTP o un proveedor equivalente.
- El resultado del envío puede registrarse para auditoría.

Diseño de componentes y clases

Componente	Tipo	Responsabilidad	Dependencia principal
RegistroResidenteView	Frontend	Formulario y retroalimentación visual	PacienteController
PacienteController	Controlador	Recibir solicitudes HTTP y delegar	PacienteService
PacienteService	Servicio	Validar y ejecutar reglas del paciente	PacienteRepository
FamiliarService	Servicio	Gestionar responsable familiar	FamiliarRepository
PacienteRepository	Datos	CRUD de pacientes	MySQL
FamiliarRepository	Datos	CRUD de familiares	MySQL
NotificacionClient	Integración	Invocar microservicio de correo	Microservicio
Paciente	Entidad	Datos del residente	-
Familiar	Entidad	Datos del responsable	-
FichaMedica	Entidad	Psicopatología y medicamentos base	Paciente

Clases principales del módulo

Paciente

Atributos: idPaciente, nombre, fechaNacimiento, idFamiliar
Métodos principales: validarDatos()

Familiar

Atributos: idFamiliar, nombre, correo
Métodos principales: validarCorreo()

FichaMedica

Atributos: idFicha, idPaciente, psicopatologia, medicinasBase
Métodos principales: actualizarFicha()

PacienteService

Atributos: No aplica directamente.
Métodos principales: registrarPaciente(), consultarPaciente(), actualizarPaciente()

PacienteRepository

Atributos: No aplica directamente.
Métodos principales: insertar(), buscarPorId(), actualizar()

Selección del módulo para UX

Se selecciona el módulo de registro de ancianos. Este módulo corresponde al proceso de gestión de internos y permite registrar al residente, asociarlo con un familiar responsable y conservar información médica inicial.

Análisis de la interfaz actual

Para efectos de la evaluación se utiliza un prototipo inicial sencillo: un formulario lineal con todos los campos en una sola vista y un único botón de guardado. Se evalúa según claridad, consistencia, visibilidad, eficiencia y retroalimentación.



Figura 1. Prototipo inicial (ANTES).

Problema	Principio afectado	Por qué afecta al usuario	Mejora propuesta
Formulario sin agrupación visual	Claridad y eficiencia	El usuario debe recorrer todos los campos sin distinguir datos personales, familiares y médicos.	Agrupar campos en secciones con títulos y jerarquía visual.
Ausencia de retroalimentación visible	Visibilidad y retroalimentación	El usuario puede no saber si faltan datos o si el registro se guardó correctamente.	Validación por campo, mensajes claros y confirmación de guardado.
Acciones poco diferenciadas	Consistencia y prevención de errores	Un único botón no comunica opciones para cancelar o volver sin guardar.	Incluir Cancelar y Guardar residente con etiquetas explícitas.

Propuesta de mejora UX

La pantalla mejorada organiza la información en tres grupos: Datos personales, Familiar responsable e Información médica. Se agregan indicaciones de campos obligatorios, validaciones y dos acciones claramente identificadas: Cancelar y Guardar residente.



Figura 2. Propuesta mejorada (DESPUÉS).

Implementación mínima

Se implementará como mínimo la agrupación visual de campos y la validación básica del formulario. El prototipo web incluido con este documento demuestra la mejora: separa los campos por secciones, marca los obligatorios, valida nombre, familiar y correo, y muestra retroalimentación al usuario. La pantalla no accede directamente a MySQL; la versión productiva enviará los datos al controlador de la capa de negocio.

Flujo técnico del registro

- El usuario completa el formulario web.
- El frontend valida formato y campos obligatorios para mejorar la experiencia.
- PacienteController recibe la solicitud.
- PacienteService vuelve a validar las reglas del negocio.
- FamiliarRepository y PacienteRepository realizan la persistencia.
- MySQL confirma la transacción.
- El controlador devuelve el resultado y la interfaz muestra confirmación o errores.

Plan de trabajo propuesto

Actividad	Duración	Dependencia	Recurso	Entregable
Revisión y validación de Fase 1	1 día	-	Analista / SQA	Requisitos validados
Diseño de arquitectura	2 días	Revisión Fase 1	Arquitecto / desarrollador	Documento arquitectónico
Diseño de clases y componentes	2 días	Arquitectura	Desarrollador	Modelo de componentes
Diseño UX del módulo	2 días	Requisitos	Diseñador / desarrollador	Bocetos antes/después
Implementación mínima web	3 días	Diseño UX	Desarrollador	Prototipo funcional
Pruebas y revisión SQA	2 días	Implementación	SQA	Evidencias y hallazgos
Correcciones y documentación final	2 días	SQA	Equipo	Entrega final

Métricas de calidad para revisión SQA

Métrica	Meta	Forma de comprobación
Trazabilidad de requisitos	100% de requisitos del módulo vinculados a diseño/prueba	Matriz de trazabilidad
Validación de campos obligatorios	100% de campos críticos validados	Casos de prueba
Errores funcionales críticos	0 al momento de entrega	Registro de defectos

Separación por capas	0 accesos directos del frontend a MySQL	Revisión de código
Retroalimentación de operaciones	100% de altas muestran éxito o error	Prueba de interfaz
Tiempo de respuesta del registro	Sin demora perceptible en carga normal académica	Prueba funcional

Seguridad considerada en el diseño

- Autenticación y autorización por roles.
- Validaciones tanto en frontend como en backend.
- Consultas parametrizadas en la capa de datos.
- Contraseñas almacenadas mediante hash seguro, nunca en texto plano.
- Comunicación web protegida mediante TLS en producción.
- Registro de operaciones sensibles para auditoría.

Conclusiones

La arquitectura propuesta mantiene una separación clara de responsabilidades y evita que la interfaz web dependa directamente de la base de datos. El módulo de registro de ancianos permite demostrar esta estructura con un caso sencillo y central para el sistema. La mejora UX propuesta reduce la carga visual, organiza la información y ofrece retroalimentación al usuario. El microservicio de notificaciones agrega una separación útil sin introducir complejidad innecesaria.

Documentación complementaria exigida para la Fase de Diseño

Esta sección desarrolla de forma explícita los entregables solicitados: planificación real para Microsoft Project, diseño arquitectónico para revisión SQA, documentación de componentes basada en clases, diseño de experiencia de usuario, definición de interfaces frontend/backend, prototipo y revisión de métricas de calidad.

Planificación real del proyecto para Microsoft Project

La planificación se organiza por actividades con duración, horas de trabajo, predecesoras, recursos, avance esperado y entregables. En Microsoft Project se recomienda utilizar calendario de lunes a viernes, jornada académica de 4 horas diarias para desarrollo y revisiones SQA en los hitos definidos. Las fechas pueden ajustarse al calendario oficial del curso sin cambiar las dependencias.

ID	Actividad	Duración	Trabajo	Predecesora	Recursos	Avance	Entregable
1	Validar requisitos de Fase 1	1 día	4 h	-	Analista, SQA	100%	Requisitos validados
2	Actualizar ERD y modelo de datos	2 días	8 h	1	Analista BD, SQA	100%	ERD ampliado
3	Diseñar arquitectura por capas	2 días	8 h	1	Arquitecto, SQA	100%	Documento arquitectónico
4	Diseñar despliegue y acceso público	1 día	4 h	3	Arquitecto	100%	Diagrama de despliegue
5	Diseñar clases y componentes	2 días	8 h	2,3	Desarrollador	100%	Diagrama de clases/componentes
6	Diseñar interfaces frontend/backend	2 días	8 h	5	Desarrollador	100%	Contratos e interfaces
7	Analizar UX del módulo de residentes	1 día	4 h	1	Diseñador UX, SQA	100%	Evaluación de usabilidad
8	Crear prototipo antes/después	2 días	8 h	7	Diseñador UX	100%	Prototipo web
9	Implementar frontend mínimo	3 días	12 h	6,8	Desarrollador	0%	Pantalla funcional
10	Implementar lógica de negocio	3 días	12 h	5,6	Desarrollador	0%	Servicios/controladores
11	Implementar acceso a datos MySQL	2 días	8 h	2,10	Desarrollador BD	0%	Repositorios/DAO
12	Implementar microservicio de correo	2 días	8 h	10	Desarrollador	0%	Servicio de notificaciones
13	Integración del módulo	2 días	8 h	9,10,11,12	Desarrollador	0%	Módulo integrado
14	Pruebas funcionales y de usabilidad	2 días	8 h	13	SQA, usuarios	0%	Evidencias de prueba

15	Revisión de métricas SQA	1 día	4 h	14	SQA	0%	Informe de calidad
16	Correcciones	2 días	8 h	15	Desarrollador	0%	Versión corregida
17	Documentación y entrega final	2 días	8 h	16	Equipo	0%	Entrega final

Duración estimada total: 32 días laborables de actividades, con tareas que pueden ejecutarse en paralelo. Trabajo estimado: 132 horas. Los porcentajes anteriores representan el estado documental actual: diseño y prototipo ya preparados; implementación e integración quedan como actividades de desarrollo.

Hitos para control con SQA

Hito	Condición de cierre	Participación SQA
H1 - Requisitos validados	Requisitos y trazabilidad revisados	Aprobar consistencia con Fase 1
H2 - Arquitectura aprobada	Capas, despliegue y dependencias definidos	Revisar separación de responsabilidades
H3 - Prototipo aprobado	Mejoras UX documentadas	Revisar criterios de usabilidad
H4 - Integración terminada	Frontend, backend, datos y correo integrados	Autorizar inicio de pruebas
H5 - Calidad aceptada	Métricas y pruebas dentro de metas	Emitir revisión final

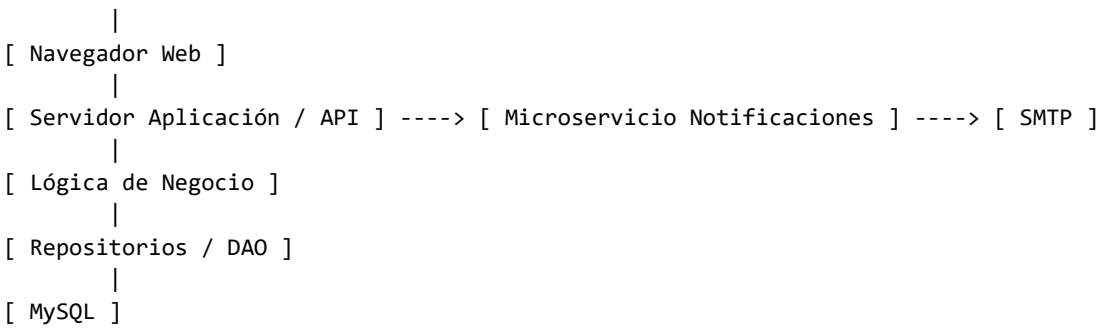
Documento de Diseño Arquitectónico para SQA

Estilo arquitectónico

Se adopta una arquitectura en capas para mantener bajo acoplamiento y responsabilidades claras. La presentación web se comunica exclusivamente con controladores/servicios de la lógica de negocio. La lógica de negocio utiliza repositorios o DAO para persistencia. Las entidades representan el dominio. MySQL permanece aislado de la interfaz de usuario.

Decisión de despliegue

Para permitir acceso desde el asilo, la fundación, médicos, laboratorio y farmacia, se propone desplegar la aplicación en un servidor en la nube con acceso público protegido por HTTPS. Esta alternativa evita depender de que una computadora doméstica permanezca encendida y facilita el acceso remoto. Para una demostración académica también puede ejecutarse localmente; la arquitectura no cambia.



Reglas arquitectónicas

- El frontend no ejecutará consultas SQL.
- Los controladores no contendrán lógica de persistencia.
- Los repositorios no implementarán reglas de presentación.
- Las reglas de negocio deberán centralizarse en servicios.
- Las credenciales y secretos no se almacenarán en el código fuente.
- Las comunicaciones públicas deberán utilizar HTTPS.
- El microservicio de correo será invocado por la lógica de negocio y no directamente por las pantallas.

Patrones de diseño empleados

Patrón	Aplicación	Beneficio
Layered Architecture	Separación frontend, negocio, datos y modelo	Reduce acoplamiento
MVC / API Controller	Controladores reciben solicitudes de la web	Separa interacción y reglas
Service Layer	PacienteService, SolicitudService	Centraliza casos de uso
Repository / DAO	PacienteRepository, FamiliarRepository	Aísla MySQL
DTO	Datos enviados entre frontend y backend	Evita exponer entidades innecesariamente
Dependency Injection	Servicios reciben repositorios por interfaz	Facilita pruebas y sustitución
Microservice	Notificaciones por correo	Aísla una función externa y sencilla

Documentación de componentes basada en clases

Capa	Clase/Interfaz	Responsabilidad	Operaciones principales
Frontend	LoginView	Capturar credenciales y mostrar resultado	submitLogin(), showError()
Frontend	RegistroResidenteView	Capturar datos del residente y familiar	validate(), submit(), showFeedback()
Frontend	ReportesView	Seleccionar filtros y visualizar reportes	filter(), export()
API	AuthController	Gestionar solicitudes de autenticación	login(), logout()
API	PacienteController	Exponer operaciones de residentes	crear(), obtener(), actualizar()
Negocio	IPacienteService	Contrato del servicio de pacientes	registrar(), consultar(), actualizar()
Negocio	PacienteService	Aplicar reglas de registro	registrar(), consultar(), actualizar()
Negocio	INotificacionService	Contrato de notificación	enviarSolicitud(), enviarCita()
Datos	IPacienteRepository	Contrato de persistencia de pacientes	insertar(), buscarPorId(), actualizar()
Datos	PacienteRepository	Implementación MySQL	insertar(), buscarPorId(), actualizar()
Datos	IFamiliarRepository	Contrato de	insertar(), buscarPorId()

		persistencia de familiares	
Modelo	Paciente	Entidad del residente	datos y relaciones
Modelo	Familiar	Responsable del paciente	datos de contacto
Modelo	FichaMedica	Información médica inicial	psicopatología, medicamentos base

Relaciones principales entre clases

```
RegistroResidenteView -> PacienteController -> IPacienteService
PacienteService implements IPacienteService
PacienteService -> IPacienteRepository
PacienteService -> IFamiliarRepository
PacienteRepository implements IPacienteRepository -> MySQL
FamiliarRepository implements IFamiliarRepository -> MySQL
PacienteService -> INotificacionService -> Microservicio de correo
```

Interfaces del frontend y backend

Interfaces de usuario del frontend

Interfaz	Usuario	Entradas	Salidas/retroalimentación
Login	Todos los usuarios	Usuario/correo, contraseña	Acceso, error o bloqueo
Registro de residente	Médico general/administración autorizada	Datos personales, familiar, información médica	Confirmación y errores por campo
Solicitudes médicas	Médico general	Paciente, motivo, especialidad, acompañante	Solicitud pendiente
Asignación de cita	Fundación	Solicitud, médico, fecha y hora	Cita asignada
Visita médica	Especialista	Diagnóstico, observaciones, exámenes, receta	Visita registrada
Caja	Caja	Tipo, paciente opcional, monto, detalle	Movimiento registrado
Reportes	Usuarios autorizados	Tipo y filtros	Vista/exportación

Contratos del backend

Endpoint lógico	Método	Responsable	Resultado
/auth/login	POST	AuthController	Sesión/token o error
/pacientes	POST	PacienteController	Paciente creado
/pacientes/{id}	GET	PacienteController	Detalle del paciente
/pacientes/{id}	PUT	PacienteController	Paciente actualizado
/solicitudes	POST	SolicitudController	Solicitud creada
/reportes/paciente/{id}	GET	ReporteController	Datos del reporte

Los endpoints anteriores son contratos de diseño; la Fase 1 no especificaba tecnología de backend ni rutas HTTP concretas. Se incluyen aquí como propuesta de diseño para implementar la separación por capas solicitada.

Experiencia de usuario: patrones y criterios

- Consistencia: mismas etiquetas, botones y ubicación de acciones en módulos similares.
- Visibilidad: el sistema debe indicar carga, éxito, error y estado de cada operación.
- Prevención de errores: validaciones antes de guardar y confirmación para acciones sensibles.
- Jerarquía visual: formularios divididos en secciones breves y comprensibles.
- Reconocimiento antes que recuerdo: menús visibles y nombres de acciones explícitos.
- Accesibilidad básica: etiquetas asociadas a campos, navegación por teclado y mensajes comprensibles.

En el módulo seleccionado se documentan como problemas principales el formulario lineal sin agrupación y la ausencia de retroalimentación visible. La mejora implementada agrupa datos personales, familiar responsable e información médica, agrega validaciones y diferencia las acciones Cancelar y Guardar residente.

Prototipo

El prototipo web entregado incluye HTML, CSS y JavaScript para demostrar la pantalla mejorada de registro de ancianos. La implementación mínima realiza validación de campos obligatorios y correo, muestra retroalimentación y mantiene la regla de que la presentación no se conecta directamente a MySQL. La integración real con controladores y servicios corresponde a la etapa de implementación.

Revisión formal de métricas de calidad SQA

Las métricas se establecen como criterios de revisión. Los resultados de ejecución se dejan pendientes hasta realizar pruebas reales; no se deben inventar valores de cumplimiento.

Métrica	Cálculo / evidencia	Meta	Responsable	Resultado actual	Estado
Trazabilidad	RF con diseño/prueba / RF totales x 100	100%	SQA	Pendiente de ejecución	Pendiente
Casos de prueba aprobados	Aprobados / ejecutados x 100	>= 95%	SQA	Pendiente de ejecución	Pendiente
Defectos críticos	Conteo de defectos críticos abiertos	0	SQA/Desarrollo	Pendiente de ejecución	Pendiente
Separación por capas	Accesos directos frontend->BD encontrados	0	SQA	Diseño cumple; código pendiente	Parcial
Validación de campos críticos	Campos validados / críticos x 100	100%	SQA	Prototipo mínimo implementado	Parcial
Retroalimentación	Operaciones con mensaje / operaciones evaluadas x 100	100%	SQA/UX	Prototipo mínimo implementado	Parcial
Usabilidad	Tareas	>= 90%	SQA/Usuarios	Pendiente de	Pendiente

	completadas / intentos x 100			prueba con usuarios	
Tiempo de respuesta	Medición de operaciones frecuentes	<= 2 s bajo carga académica normal	SQA	Pendiente de ejecución	Pendiente

Lista de verificación SQA

☐ Los requisitos de Fase 1 tienen trazabilidad hacia diseño y pruebas.

☐ La interfaz web no accede directamente a la base de datos.

☐ Los servicios contienen las reglas de negocio.

☐ Los repositorios encapsulan la persistencia.

☐ Los permisos por rol se validan en backend.

☐ Los formularios muestran errores comprensibles.

☐ Las operaciones sensibles quedan preparadas para auditoría.

☐ Las copias de seguridad y recuperación están documentadas.

☐ El prototipo antes/después cuenta con evidencia visual.

Matriz de cumplimiento de la consigna

Requisito	**Evidencia en este documento**
Planificación Microsoft Project	Sección 15: actividades, duración, trabajo, dependencias, recursos, avance e hitos SQA.
Diseño arquitectónico	Sección 16: capas, nube/acceso público, despliegue, reglas y patrones.
Componentes basados en clases	Sección 17: clases, interfaces, responsabilidades y relaciones.
UX y patrones	Sección 19 y prototipos antes/después incluidos previamente.
Interfaces frontend/backend	Sección 18: pantallas y contratos lógicos del backend.
Prototipo	Sección 20 y prototipo web entregado.
Métricas SQA	Sección 21: métricas, metas, responsables y checklist.

Observaciones de alcance

La documentación distingue entre lo ya definido en la Fase 1 y las decisiones propuestas para la Fase 2. La Fase 1 estableció la aplicación web modular, MySQL, roles, requerimientos, casos de uso y un ERD inicial. Las rutas HTTP, interfaces de programación, despliegue en nube y patrones específicos se documentan en esta fase como decisiones de diseño. Los resultados de pruebas SQA permanecen pendientes hasta ejecutar la implementación completa.